Computer Organization and Architecture
(Code: CS256)

Lecture-23: Cache Write

Course Faculty

Dr. B. R. Bhowmik

Assistant Professor

Dept. of CSE

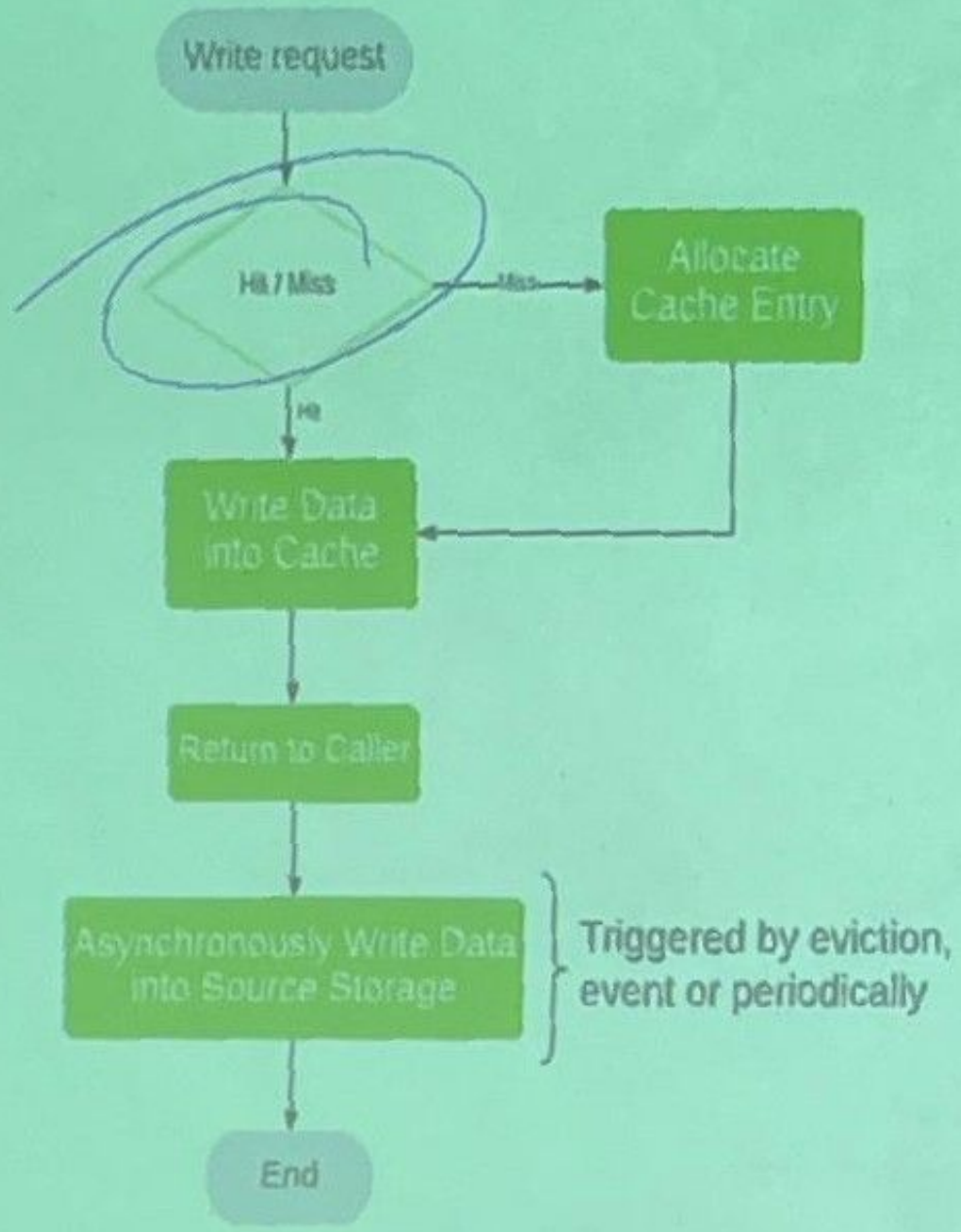
Website: https://sites.google.com/view/biswajitrbhowmik
https://sites.google.com/view/vicayana-gaveshana

Course Email: sangsangstha@gmail.com

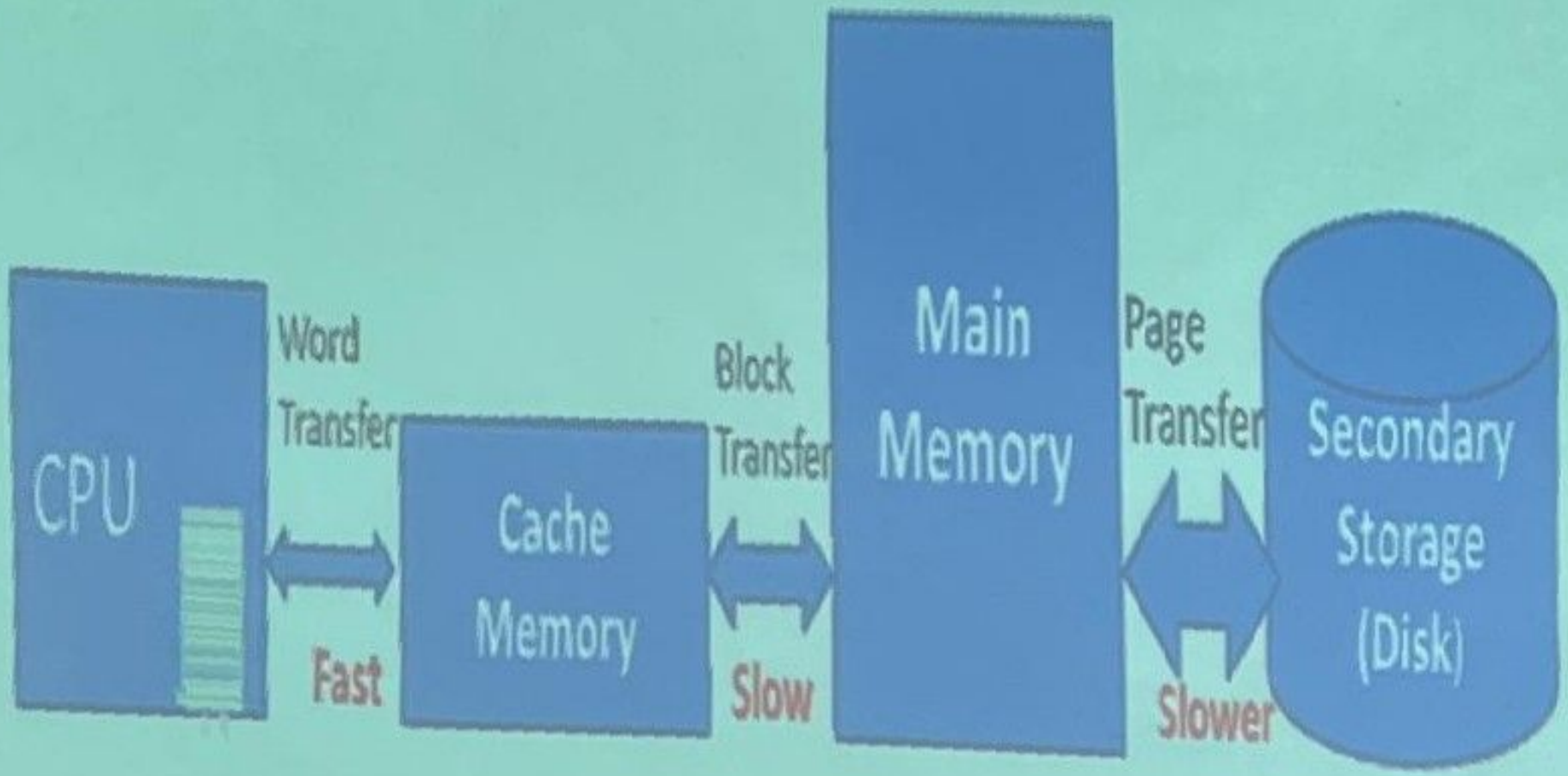
Elements of Cache Design

- Size
  - Cache Size,
  - Block Size
  - Line Size
- Addressing
  - Logical
  - Physical
- Mapping Function
  - Direct,
  - Associative,
  - Set Associative
- Write Policy
  - Write through,
  - Write back,
  - Write once
- Number of Caches
  - Single or Multi level,
  - Unified or Split
- Replacement Algorithms
  - Least recently used (LRU),
  - First in first out (FIFO),
  - Least frequently used (LFU),
  - Random

Cache Write


Write request
Hit / Miss
Miss
Allocate Cache Entry
Hit
Write Data into Cache
Return to Caller
Asynchronously Write Data into Source Storage
Triggered by eviction, event or periodically
End

CPU-Cache-Memory



Cache *WRITE* Operation

- Before we write a value, we need to ensure that the block is present in the cache.
- Why ?
  - Otherwise, we have to maintain the indices of the bytes that were written to
  - We treat a block as an atomic unit
  - Hence, on a miss, we fetch the entire block first
- Once a block is there in the cache, Go ahead and write to it .
- Maintain a modified bit in the tag array.

Solution ???

Cache Write Policy

Cache Write Policy

Cache Write Policy

- *A cache's write policy is the behavior of a cache while performing a write operation.*
- A cache's write policy plays a central part in all the variety of different characteristics exposed by the cache.
- *It gives the different ways to implement write operations in a cache.*

Cache Write Policy

- Cache Write Policy I- on Write Hit
- Cache Write Policy II – on Write Miss

Mr. Hit

Ms. Miss

Cache Write Policy I

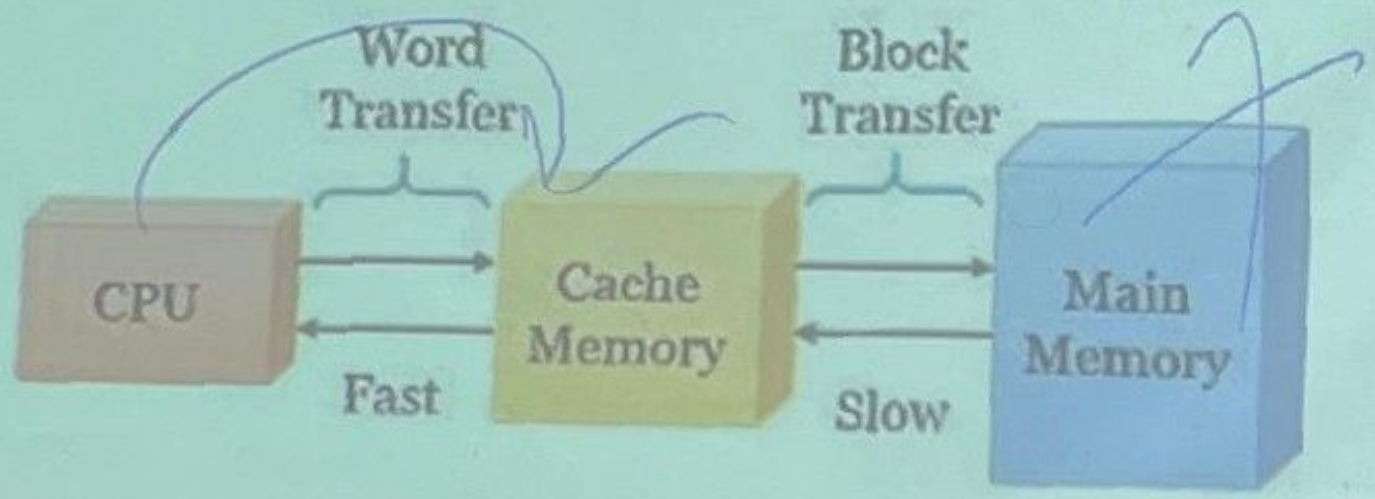
- Data write operation on write hit follows following write policies.
  - Write through / Store through
  - Write back / Copy back
  - Write Once (Cache coherence)

Cache Write Policy II

- What happens on write-miss
  - Write Allocate
  - Write Non-Allocate or write-around

Cache Write Issues

- We can perform both read operation and write operation on the cache memory.
- When we perform read operation on cache memory then it does not change the content of the memory.
- What happens when there is write request for a block.
- Blocks in main and cache memory may differ.
- Memory writes = 15% of total instructions



Cache Write Issues

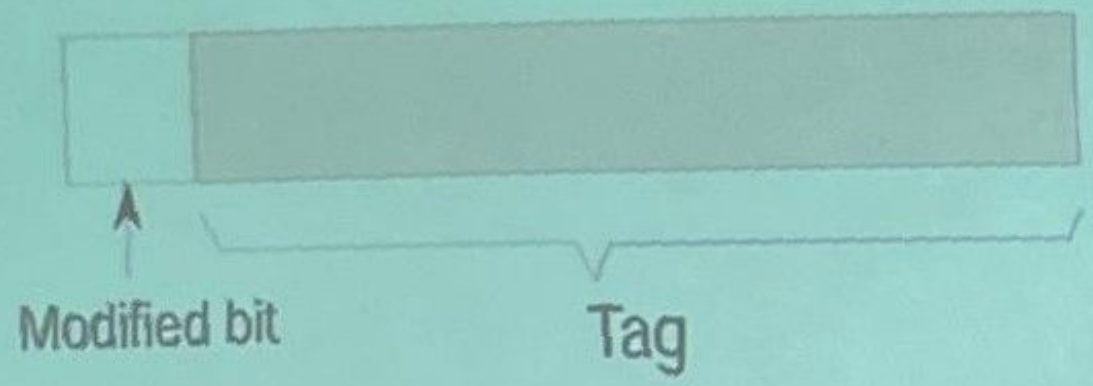
- Must not overwrite a cache block unless main memory is up to date
- Multiple CPUs may have individual caches
- I/O may address main memory directly

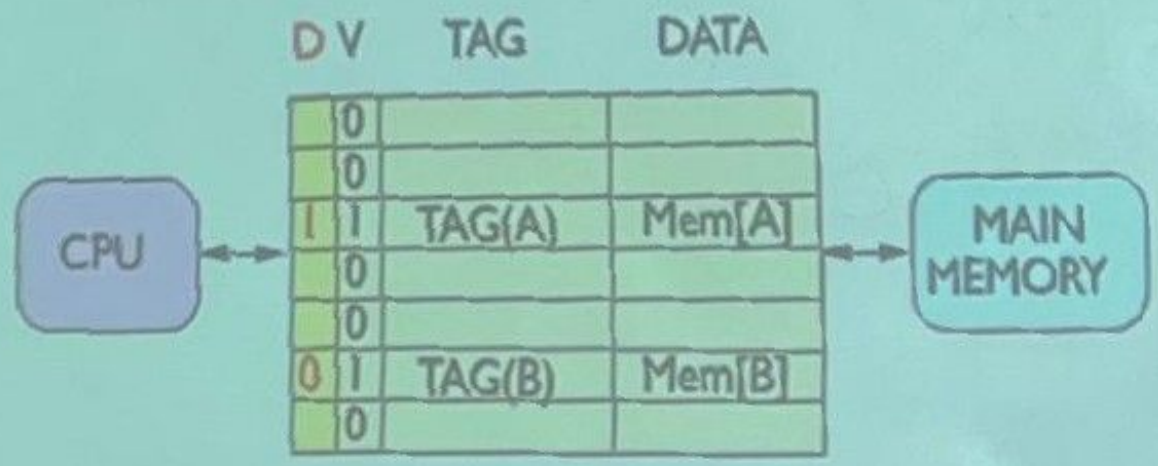
Dirty Bit

- Each Block in the cache needs a bit to indicate if the data present in the cache was modified (Dirty) or not modified (Clean).
- It is status bit related to a cache block
- **Dirty bit** → 1
- **Clean bit** → 0
- If it is clean there is no need to write it into the memory.
- It is designed to reduce write operation to a memory.

Dirty Bit

- If a block has been written to, after it was fetched, set it to 1.





Dirty Bit

Read from address 1010000001100000

1010	0000011	00000
Tag	Index	Offset

Direct-Mapped Cache

Line #	V	D	Tag	Data (32-byte block)
0:	0	0		
1:	1	1	1111	
2:	0	0		
3:	1	0	1010	
4:	0	0		
	...			
127:	0	0		

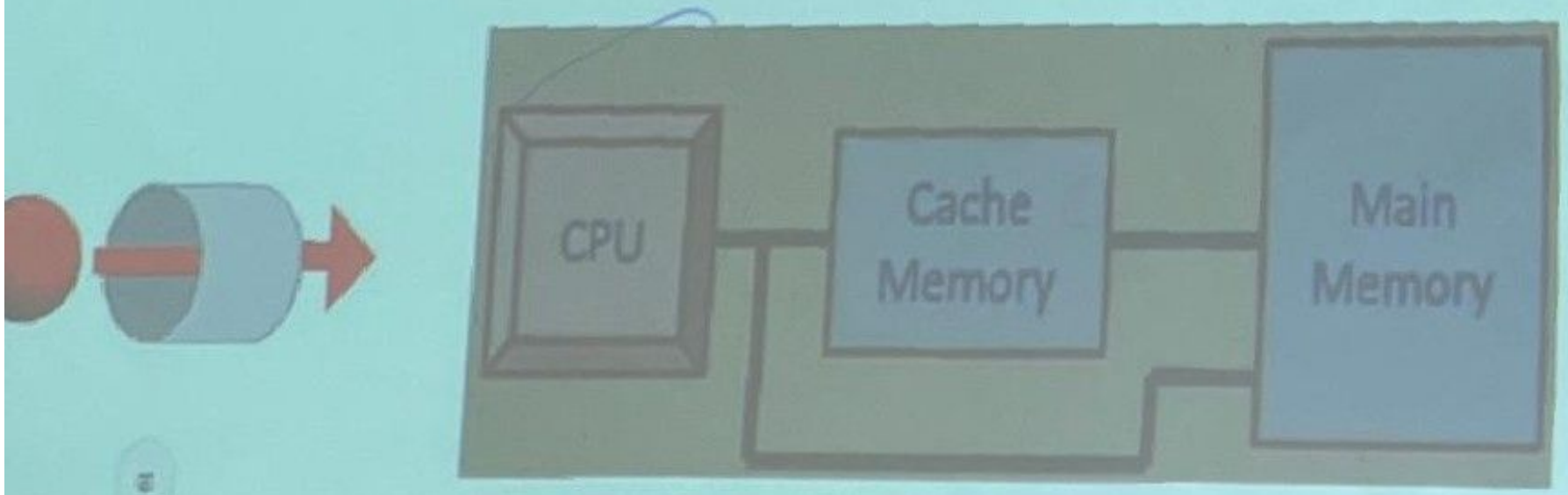
Result: hit, the line is valid, and the tag matches.

Cache Write Policy I

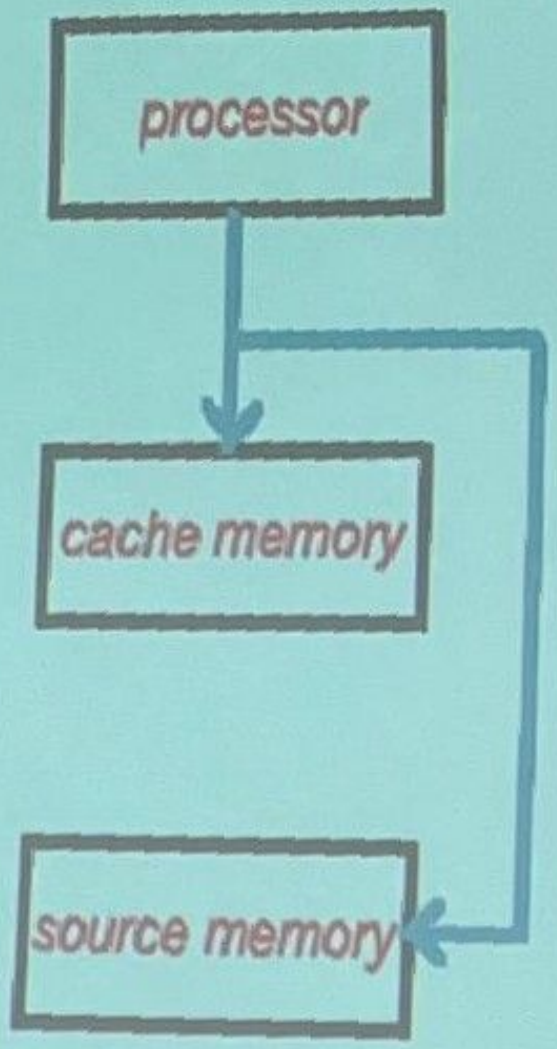
1. Write Through / Store through
2. Write Back / Copy back
3. Write Once (Cache coherence)

Write Through Policy

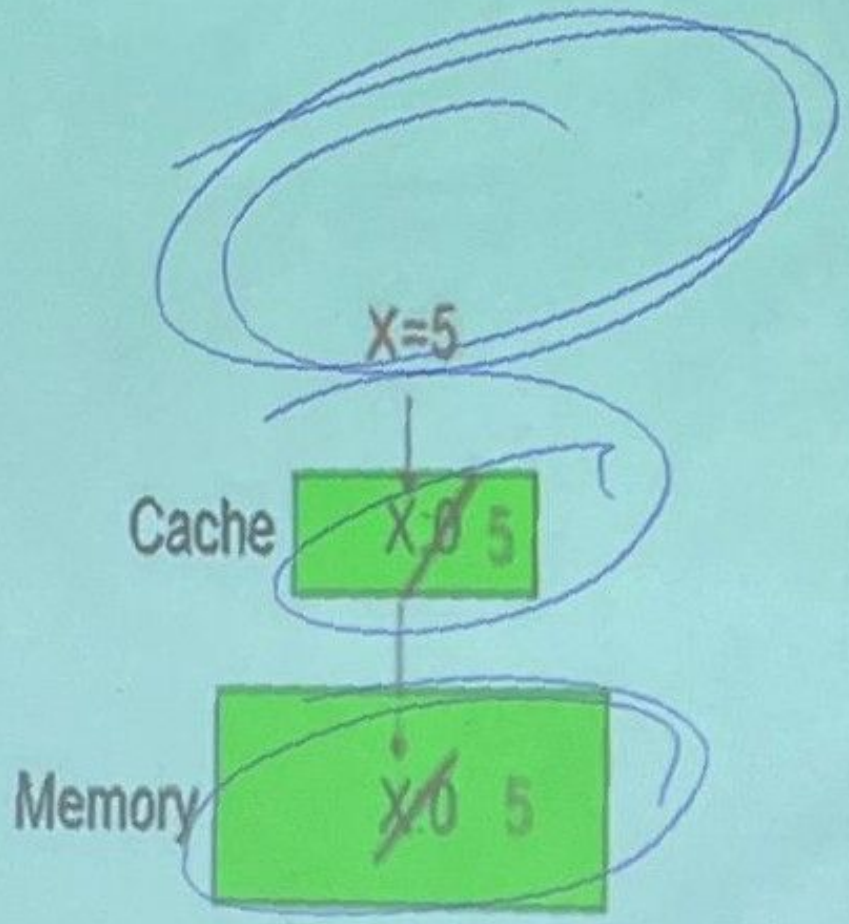
- *All writes go to main memory as well as cache.*
- This process is simpler and more reliable.
- *This is used when there are no frequent writes to the cache.*
- Multiple CPUs can monitor main memory traffic to keep local (to CPU) cache up to date.
- It is two types- ***through without/with buffer*** write



Write Through Policy



write-through



Write Through Example

	V	TAG	DATA
100	1	10101	0303

MAIN MEMORY

1010 1100	0303

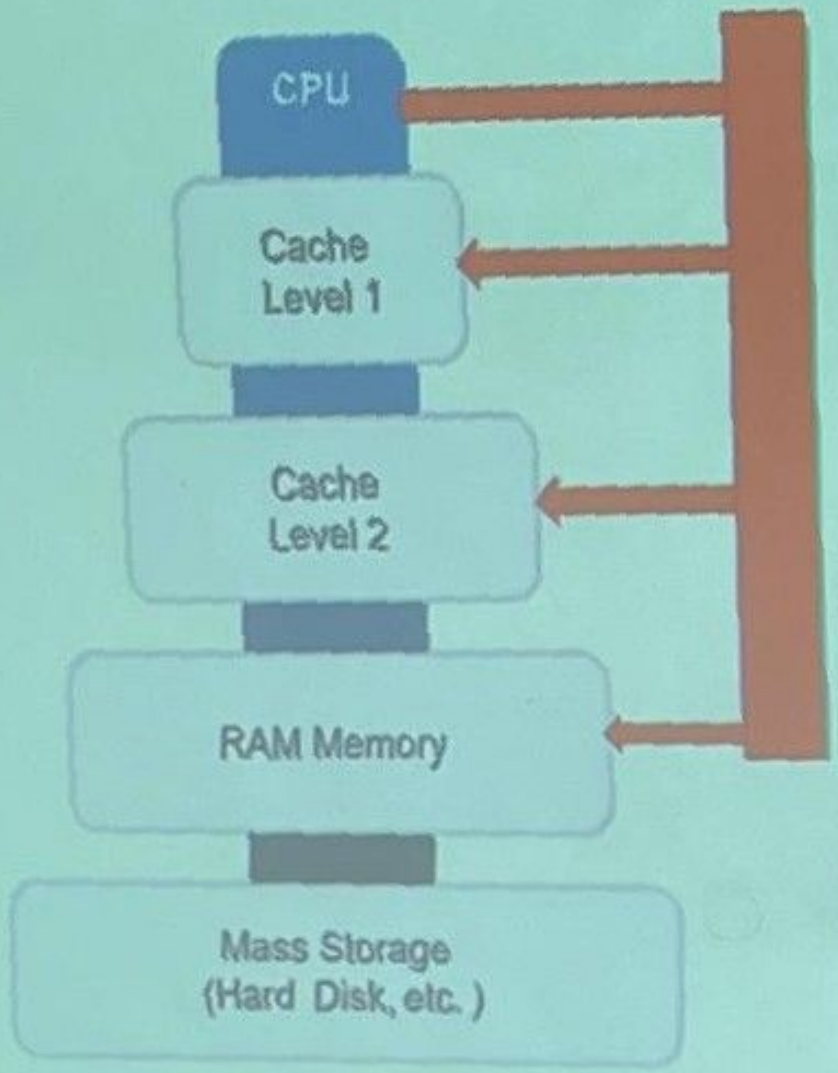
AFTER
WRITE THROUGH

	V	TAG	DATA
100	1	10101	2203

MAIN MEMORY

1010 1100	2203

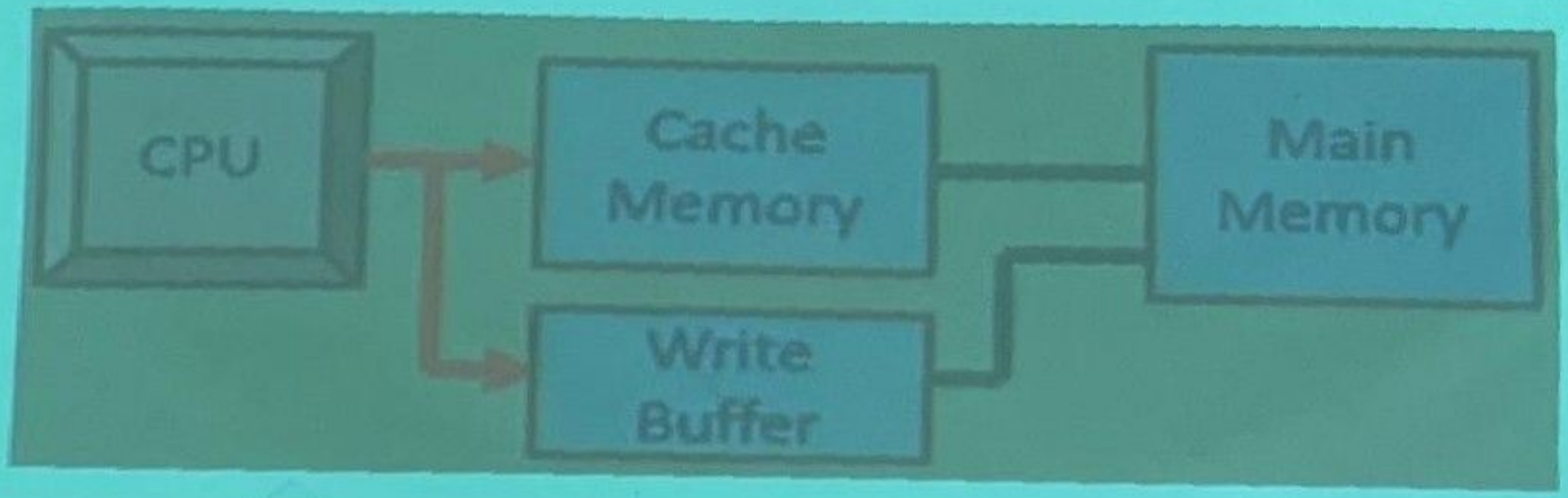
Write Through Policy



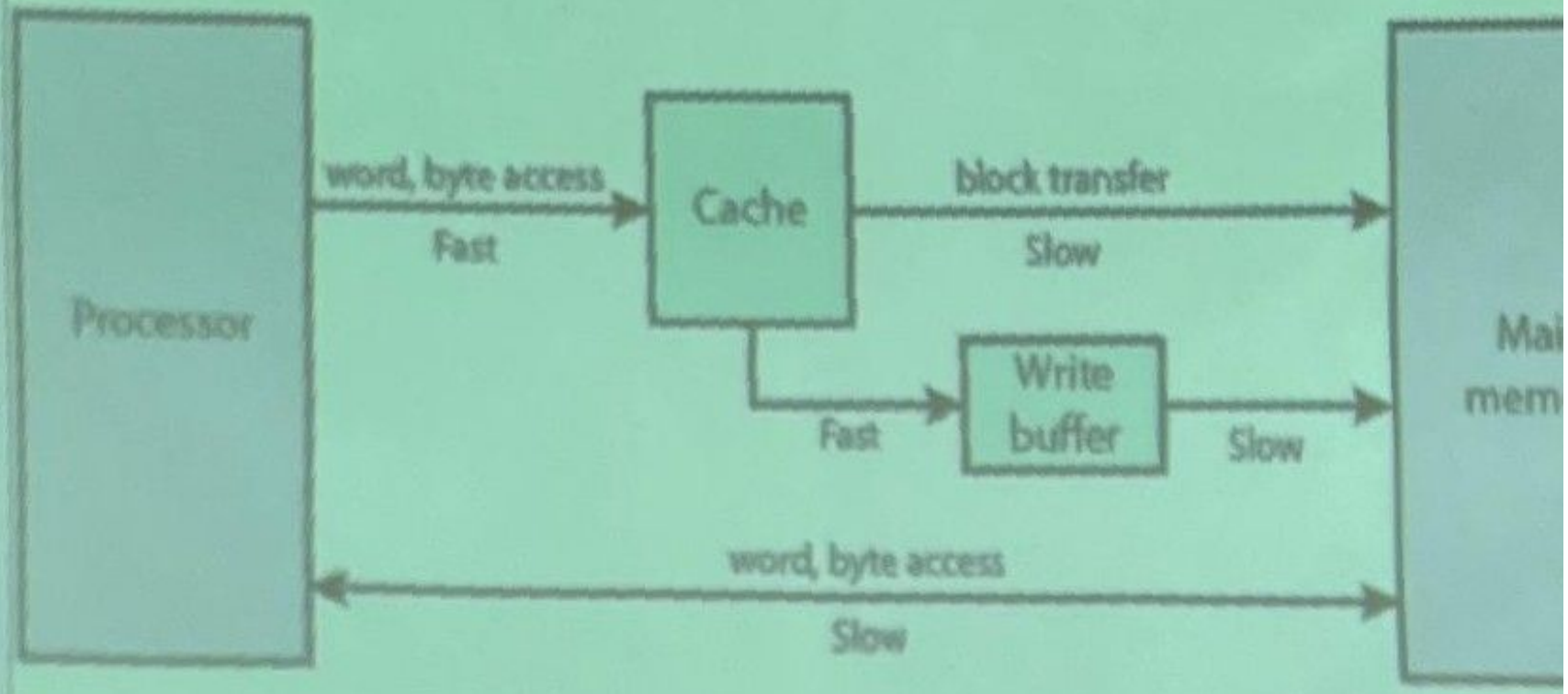
Write Through via Buffer

- **Perfect Write Buffer:**
  - All writes are handled by write buffer
  - No stalling for write operations.

- **Realistic Write Buffer:**
  - Percentage of write stalls are not eliminated when the write buffer is full.



ARM Cache and Write Buffer Organizati

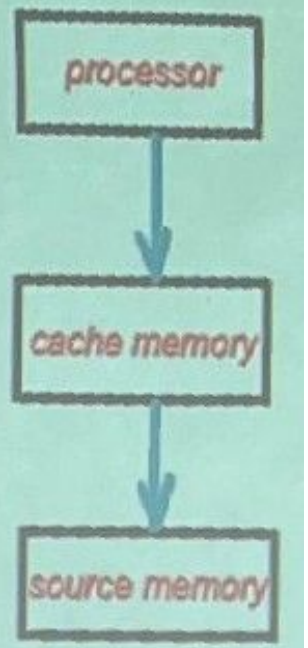


Write Through Policy

- Disadvantages:
  - Lots of traffic
  - Slows down writes
  - Remember bogus write through caches!
- Use of **write buffer**.
  - ➢ Used to reduce CPU write stall time when data is written to main memory.

Write Back Policy

- *Updates initially made in cache only.*
- Update bit for cache slot is set when update occurs.
- *If a block is to be replaced, write to main memory only if update bit is set.*
- Write-back cache blocks can be clean or dirty.
- A status bit called dirty or modified bit is associated with each cache block.
- The bit indicates whether a cache block is modified.
- *Write Back is also known as* ***Write Deferred or Copy Back.***



write-back

Write Back Policy

- Advantages
  - Write occurs at the speed of cache memory.
  - Multiple writes to a cache block requires only one write to main memory.
  - Needs less memory bandwidth
  - Attract multi-processing systems
- Disadvantages
  - Other caches get out of synchronization.
  - Cache coherence problem in multi-processing systems.
  - I/O must access main memory through cache.

Write Back Example

	V	TAG	DATA
100	1	10101	0303

MAIN MEMORY

1010 1100	0303

AFTER WRITE BACK

	V	TAG	DATA
100	1	10101	2203

MAIN MEMORY

1010 1100	0303

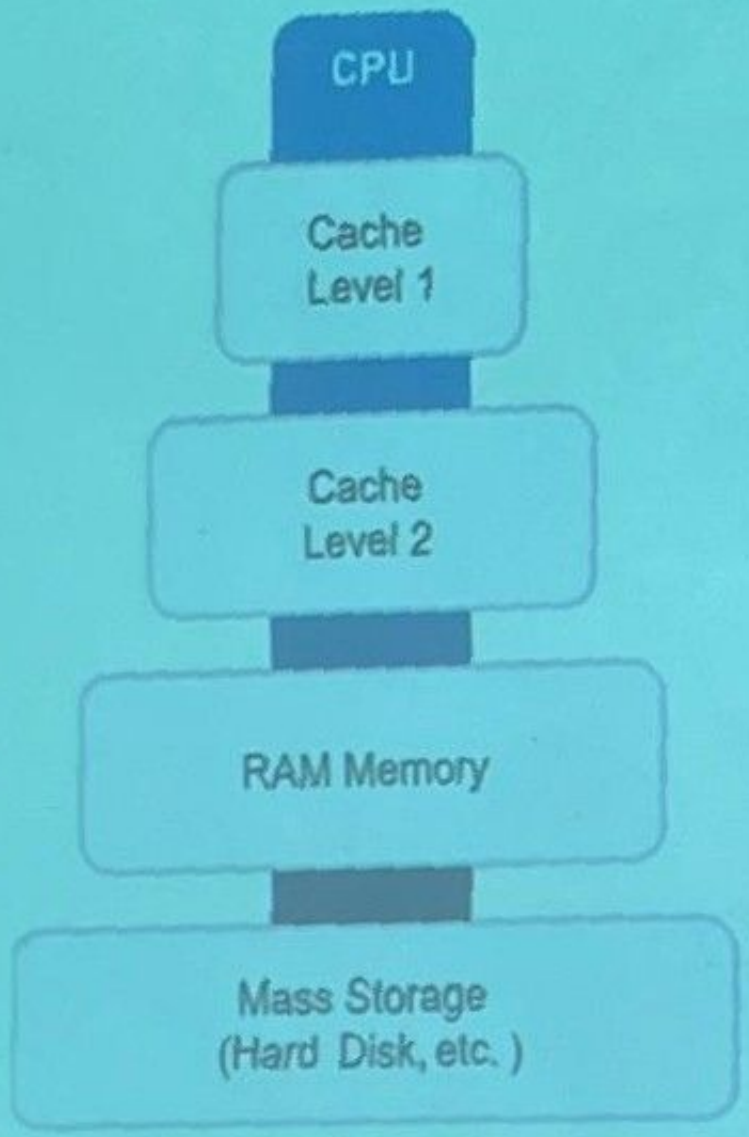
WHEN THE CACHE LINE IS ABOUT TO BE REPLACED
THE DATA WILL BE UPDATED

	V	TAG	DATA
100	1	SOME REPLACED DATA	

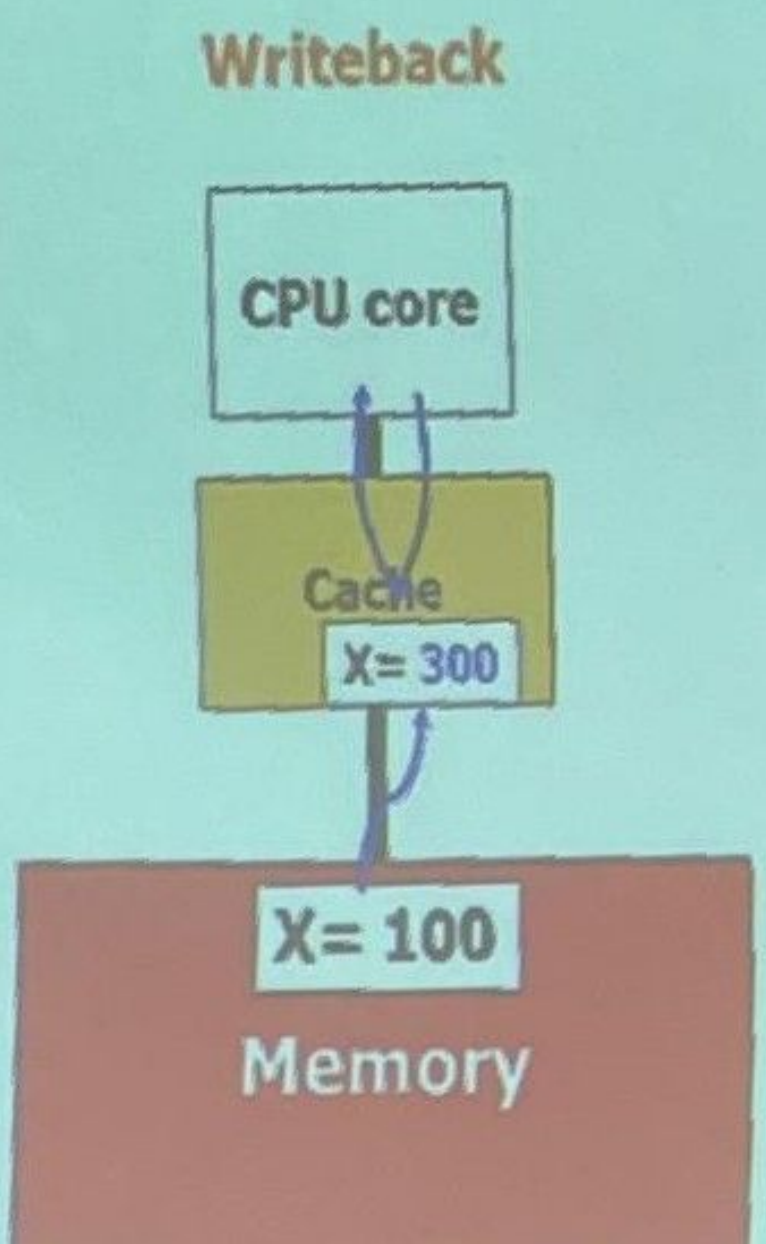
MAIN MEMORY

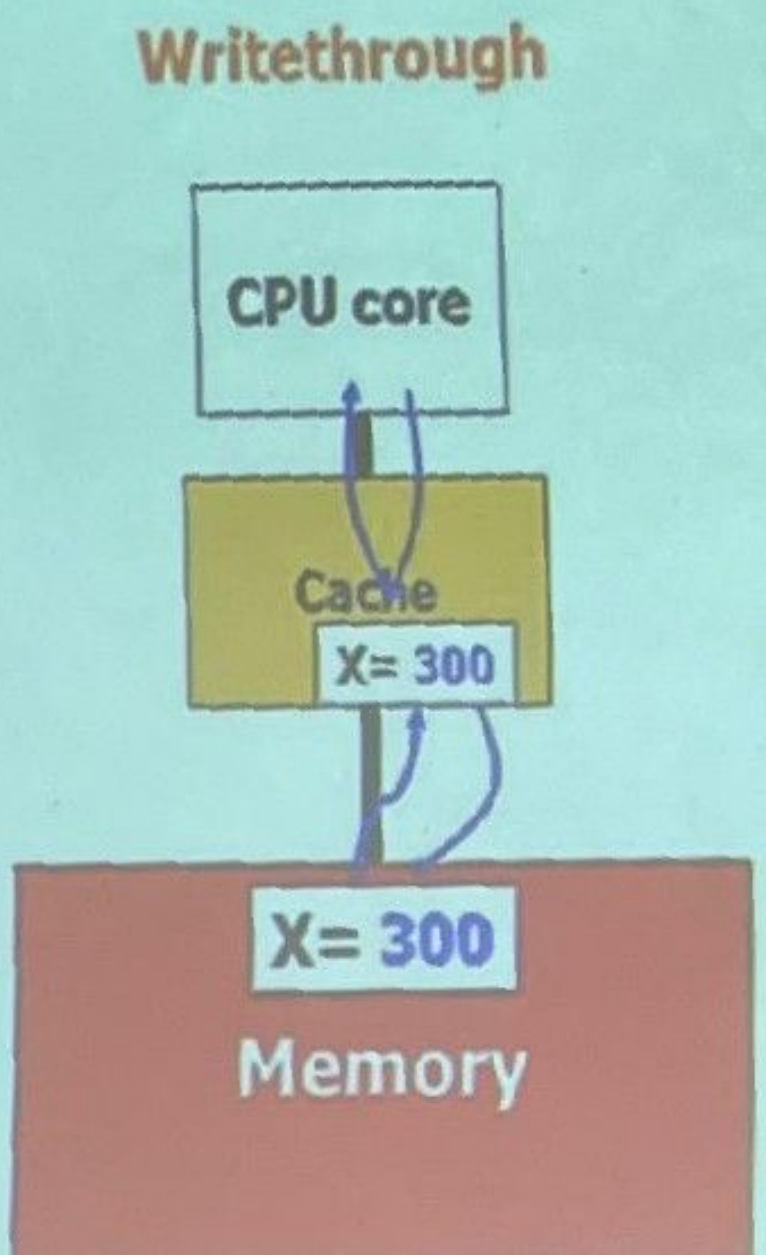
1010 1100	2203

Write Back

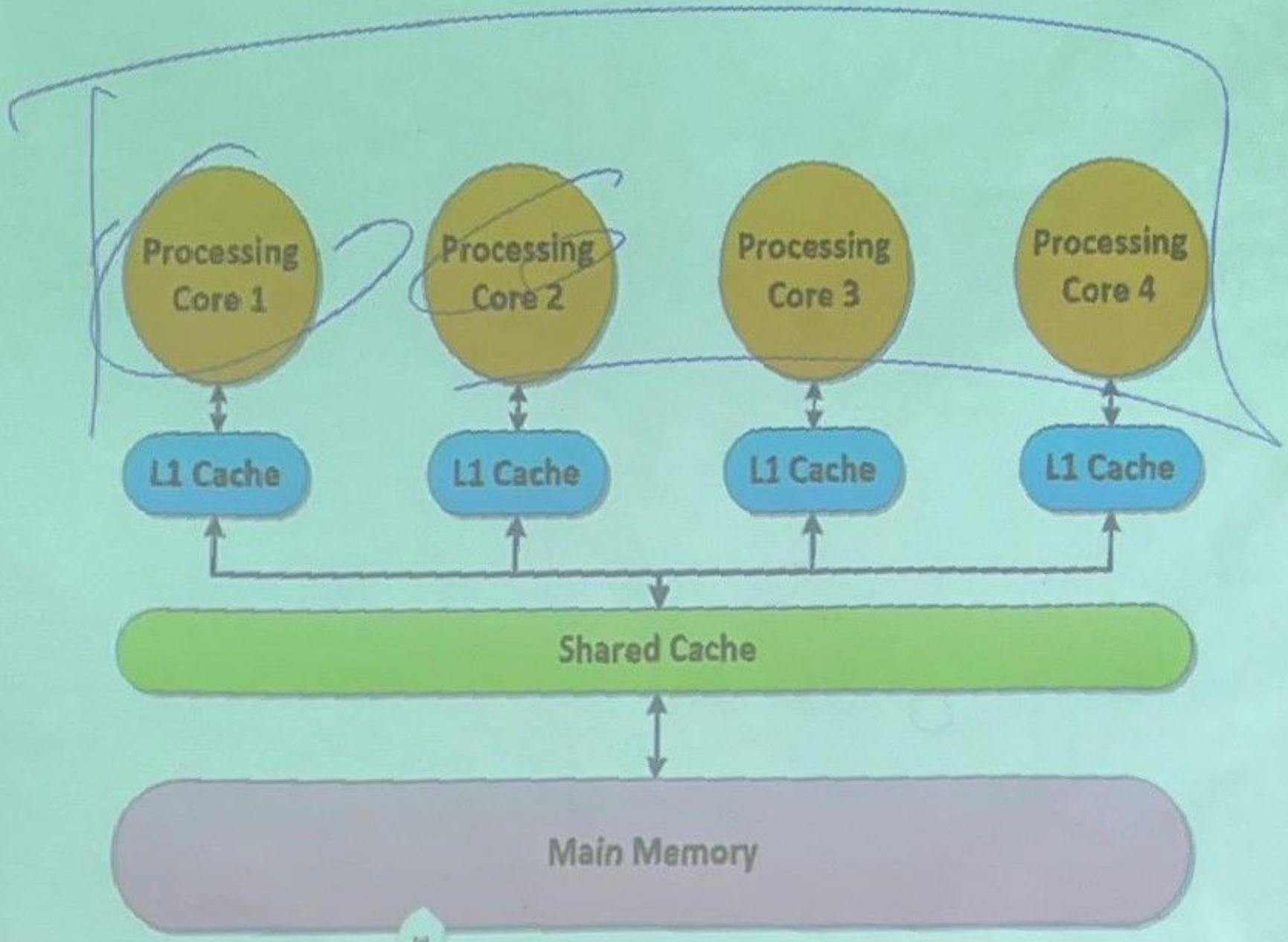

CPU
Cache
Level 1
Cache
Level 2
RAM Memory
Mass Storage
(Hard Disk, etc.)

Write Through and Write Back



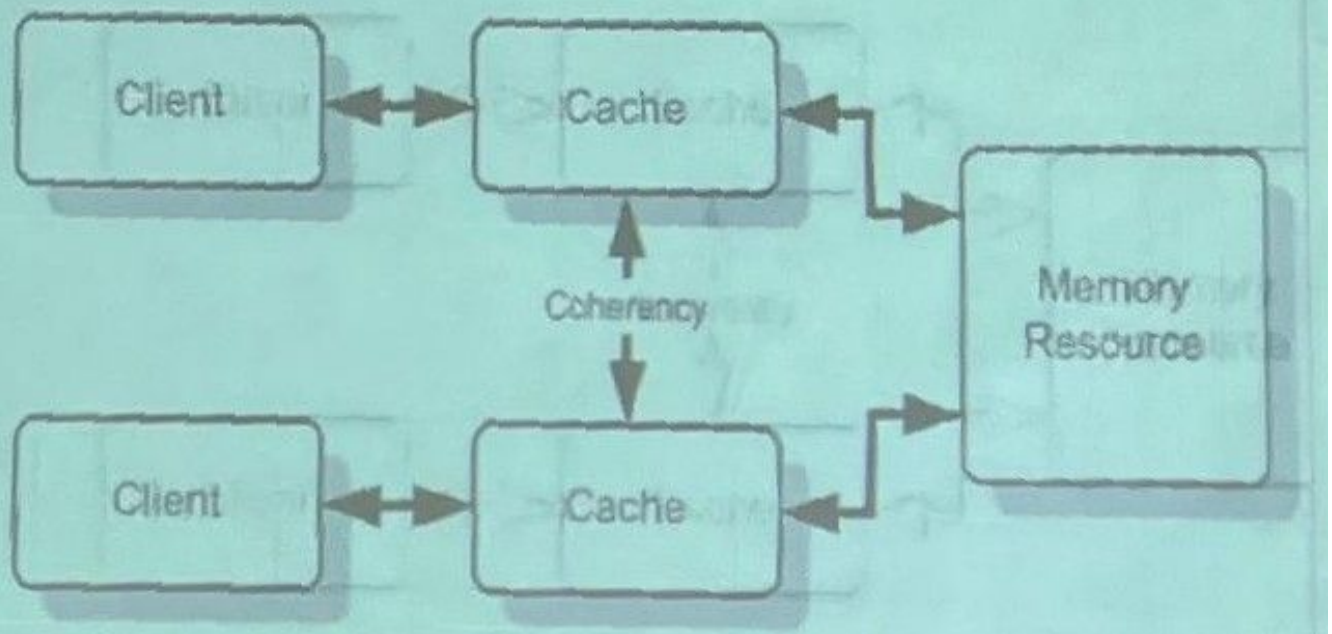


Multi-Core Systems

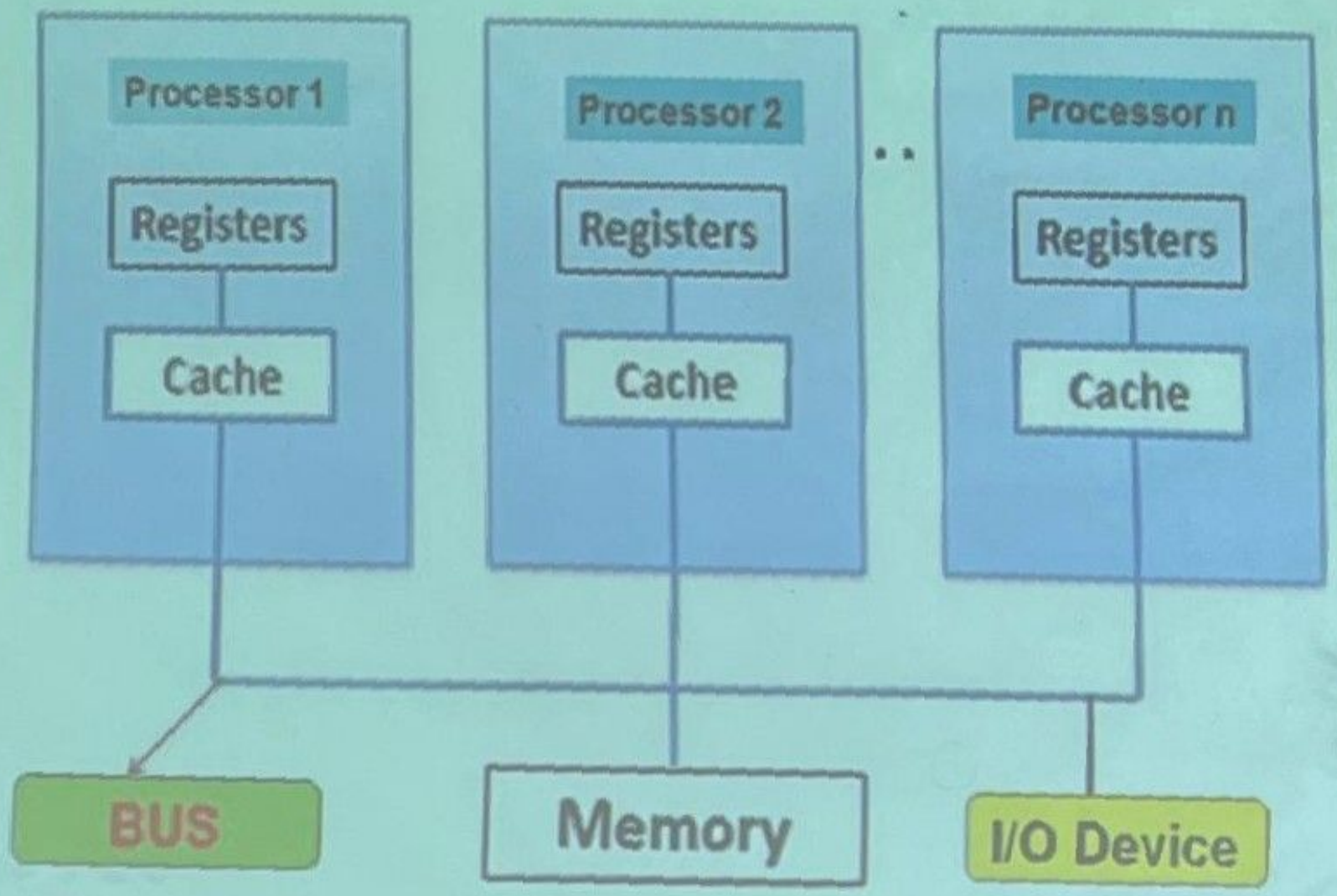


Write Once

- Cache coherence is the uniformity of shared resource dat that ends up stored in multiple local caches.
- When clients in a system maintain caches of a common memory resource, problems may arise with incoherent d which is particularly the case with CPUs in a multiproces system.



Multi-Processing Systems



MESI Protocol

- When the block is marked M (modified) or E (exclusive), the copies of the block in other Caches are marked as I (Invalid).

	M	E	S	I
M	✗	✗	✗	✓
E	✗	✗	✗	✓
S	✗	✗	✓	✓
I	✓	✓	✓	✓

Write-Once

- Read hit : Data is in cache, success!
- Read miss :
  - If no other cache has a DIRTY copy, then read from memory, state set to RESERVED
  - If any cache has a DIRTY copy, that copy is supplied, and is also written back to memory, all copies' state set to VALID

Write-Once

- Write Hit:
  - If the block state is DIRTY, write proceeds locally in cache
  - If the block state is RESERVED, write proceeds locally in cache, but state is set to DIRTY
  - If the block state is VALID, word is written through to memory, and state is changed to RESERVED.
  - Other caches with the same block observe this bus write, and change their states to INVALID

Write-Once

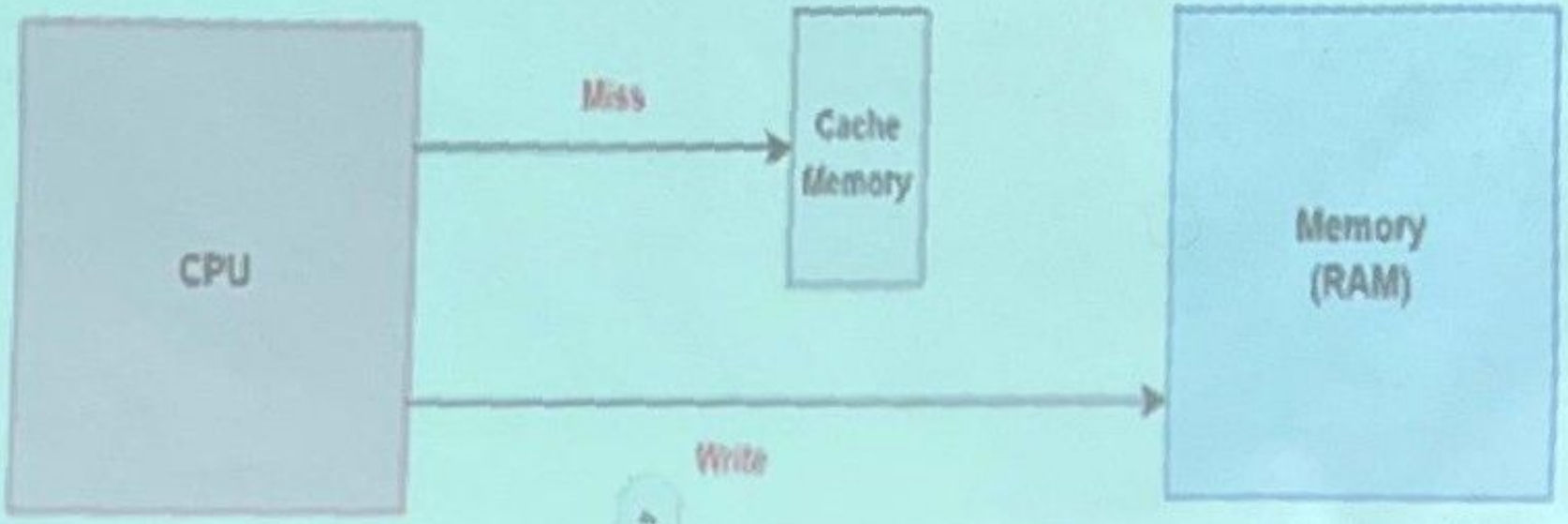
- Write Miss:
  - If no other cache has a DIRTY copy, then read from memory, state set to RESERVED
  - If some cache has a DIRTY copy, the block is loaded in the from the cache with DIRTY copy, and that cache invalidates
  - Then, proceed with write, and the state is set to DIRTY

Cache Write Policy III

1. Write Allocate
2. Write Non-allocate (or "write-around"):

Write Policy II

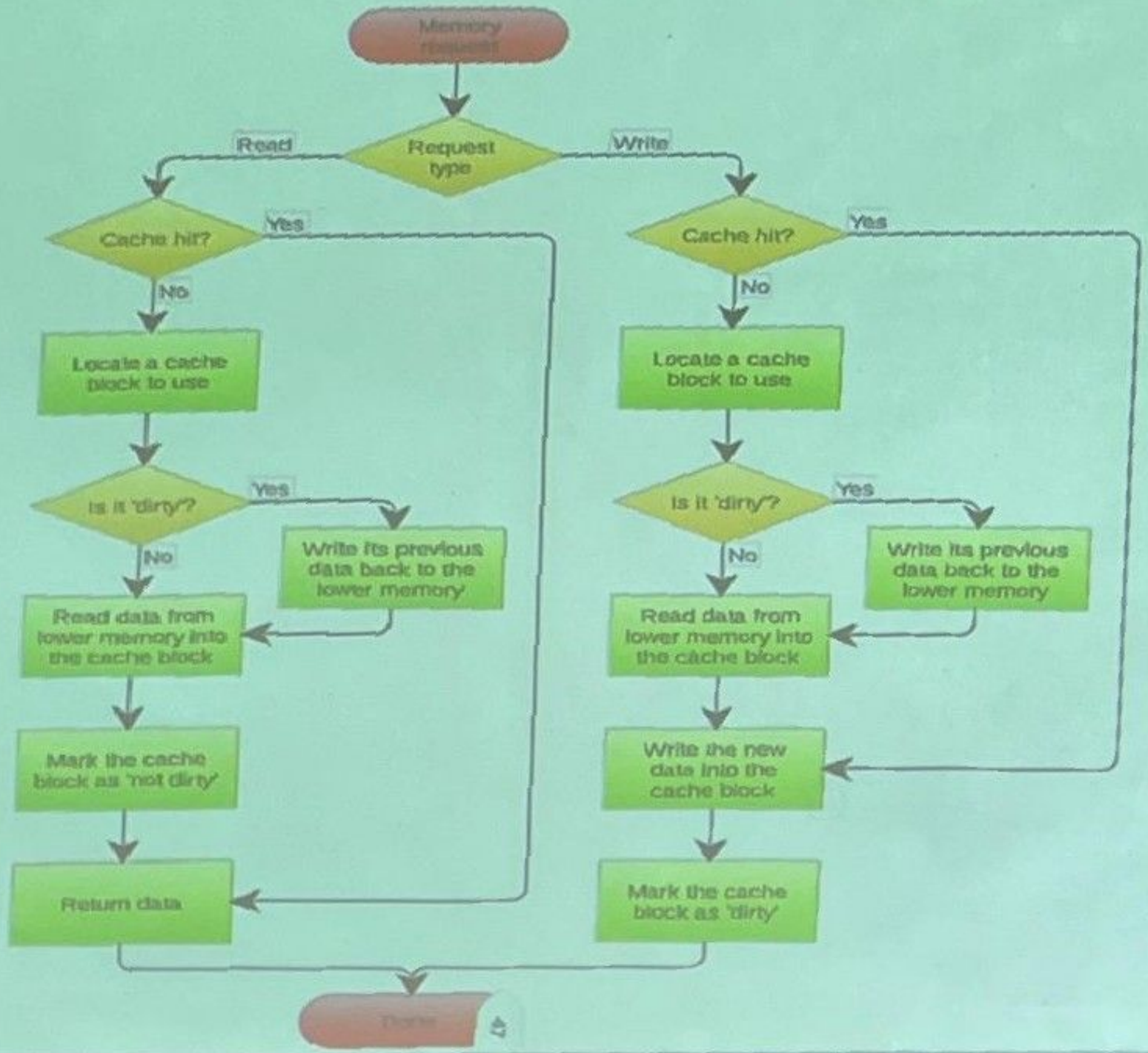
- What happens on write-miss?
- Solutions
  - Write allocate
  - Write non-allocate (or "write-around"):



Write Allocate

- *A write-allocate cache makes room for the new data on a write miss, just like it would on a read miss.*
- *Allocate new cache line in cache.*
- Usually means that you have to do a "read miss" to fill in rest of the cache-line!
- Alternative: per/word valid bits
- *Write allocation works with both Write back and Write through.*
- But it is generally used with Write Back because it is unnecessary to bring data from the memory to cache and then updating the data in both cache and main memory.

Write Back Write Allocate



Write Back Write Allocate

DATA 2203 IS TO BE STORED IN LOCATION
1010 1100 ON A WRITE MISS

	V	TAG	DATA
100	0		

MAIN MEMORY

1010 1100	0 3 0 3

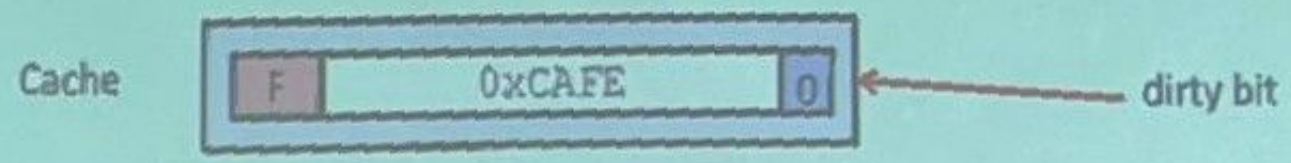
AFTER
WRITE ALLOCATE

	V	TAG	DATA
100	1	10101	2 2 0 3

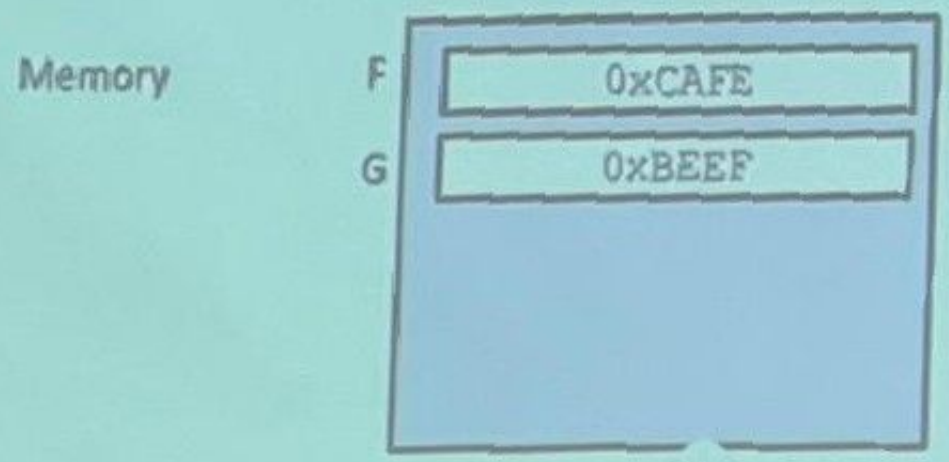
WILL BE UPDATED LATER IF WRITE BACK AND IMMEDIATELY IF WRITE THROUGH

Write Back Write Allocate

```
mov 0xFACE, F
```

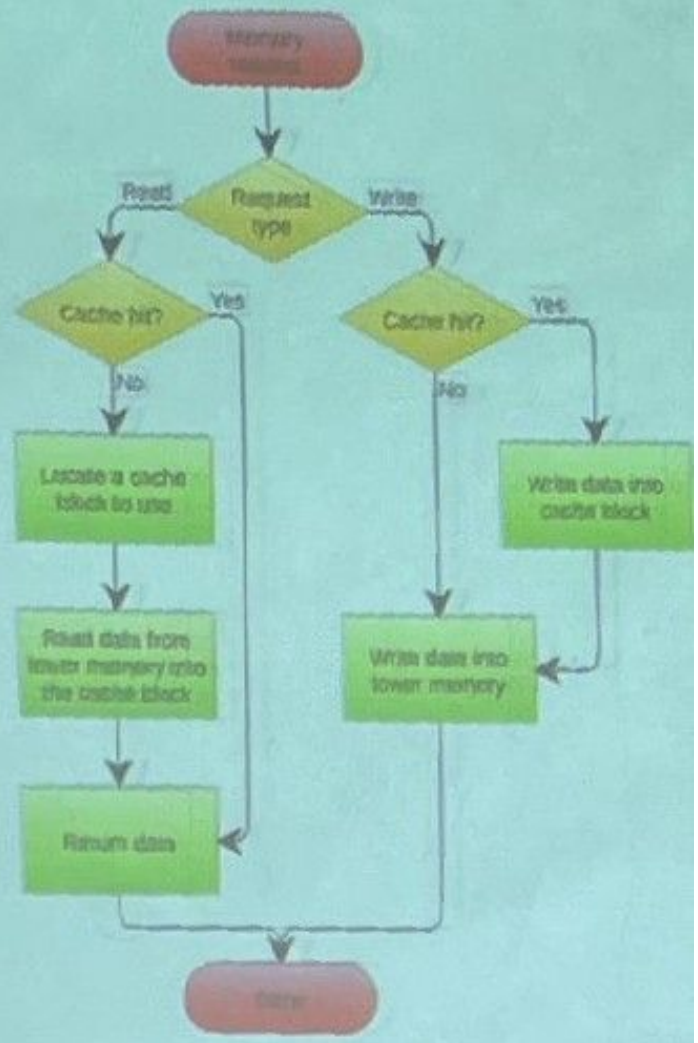


Step 1: Bring F into cache



Write Non-Allocate

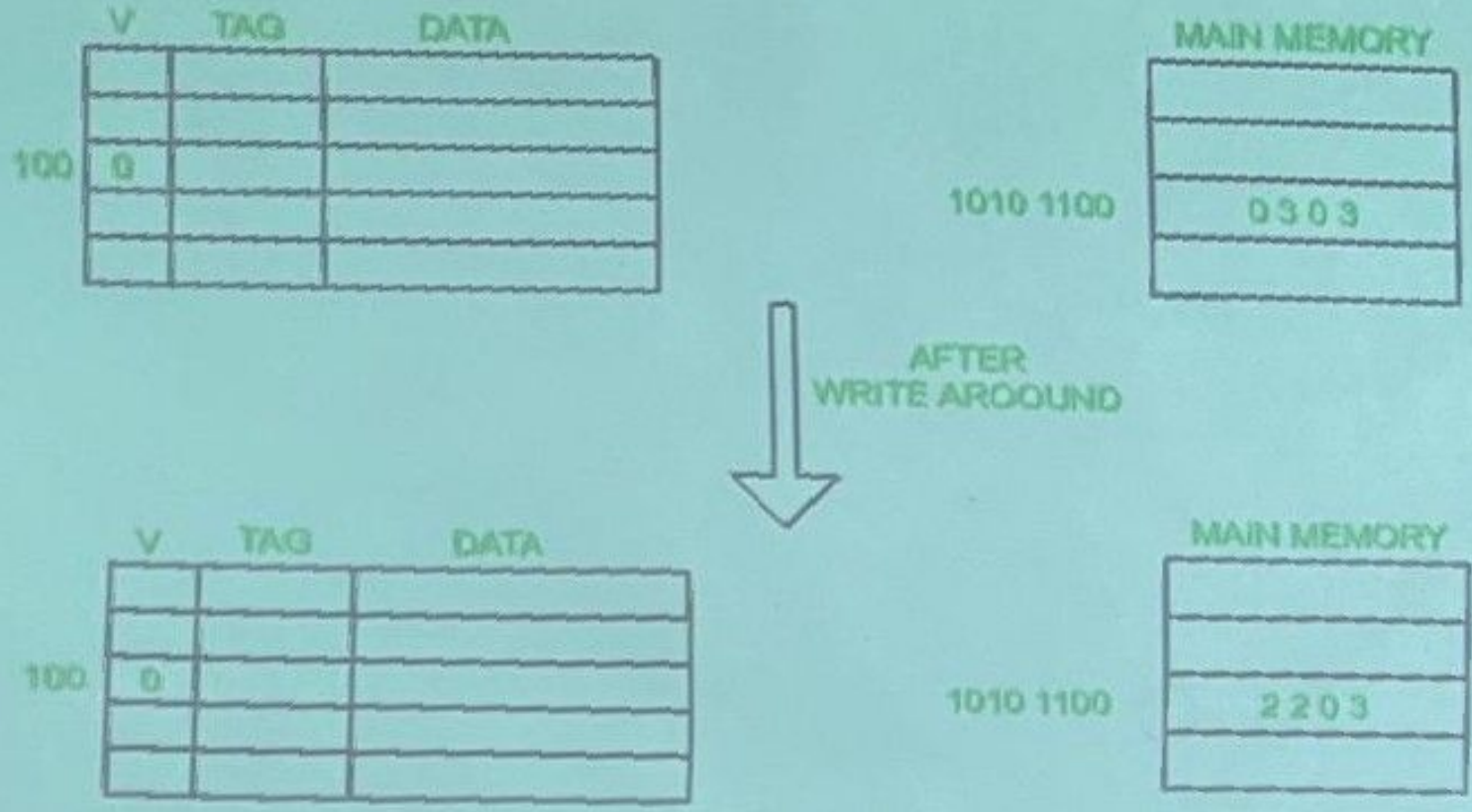
- Simply send write data through to underlying memory/cache - don't allocate new cache line!
- *Write Through is often used with No write Allocate.*
- Data is Directly written/updated to the main memory without disturbing the cache.
- It is better to use this when the data is not immediately used again.



Write through write non allocate

Write Through Write Non-Allocate

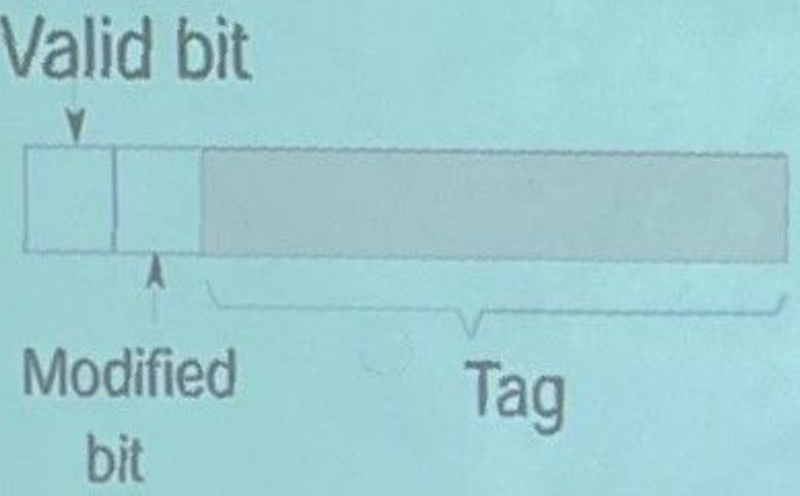
DATA 2203 IS TO BE STORED IN LOCATION
1010 1100 ON A WRITE MISS



Insert Operation

If we don't find a block in a cache. We fetch it from the lower level. Then we insert the block in the cache → insert operation

- Let us add a valid bit to a tag
  - If the line is non-empty, valid bit is 1
  - Else it is 0
- Structure of a tag



Insert Operation

- Check if any way in a set has an invalid line
  - If there is one, then write the fetched line to that location, set the valid bit to 1.
- Otherwise, find a candidate for replacement.

evict Operation

- If the cache is write-through
  - Nothing needs to be done
- If the cache is write-back
  - AND the modified bit is 1
  - Write the line to the lower level